

[YOUR COLLEGE / UNIVERSITY NAME]

[DEPARTMENT OF COMPUTER SCIENCE]

[CITY]

[COURSE NAME]

A PROJECT REPORT ON

**“SOCIALKARO – SOCIAL MEDIA MARKETING
MANAGEMENT SYSTEM”**

(A simple file-based system using Bash Shell Script)

IN PARTIAL FULFILLMENT OF

BACHELOR OF [YOUR PROGRAM]

SEMESTER [X] – [ACADEMIC YEAR]

UNDER THE GUIDANCE OF

[GUIDE NAME]

SUBMITTED BY: [YOUR NAME]

ROLL NO: [YOUR ROLL NUMBER]

[YOUR COLLEGE / UNIVERSITY NAME]

[DEPARTMENT OF COMPUTER SCIENCE]

Project Completion Certificate

THIS IS TO CERTIFY THAT

[YOUR NAME] of [YOUR CLASS / DIVISION] has completed the project titled
“SOCIALKARO – SOCIAL MEDIA MARKETING MANAGEMENT SYSTEM” under
our guidance and supervision during the academic year [ACADEMIC YEAR] in the
department of [DEPARTMENT NAME].

Project Guide

Head of Department

External Examiner

Introduction

SOCIALKARO is a simple command-line Social Media Marketing Management System written as a Bash shell script. A customer can register, log in, see the list of marketing services, place an order, pay for it and get a receipt.

The system does **not use any database**. All records are kept in **three plain text files** – one for customer data, one for customer login credentials and one for receipts. Whenever a customer enters their details, the script checks them against these files using **grep**.

Project Overview

The system follows a simple customer journey:

**Main Menu → Register / Login → Customer Menu → My Details → Place Order & Pay → Receipt
→ My Receipts → Change Password → Logout**

The whole program uses only the Linux commands and shell programming topics from the syllabus: echo, printf, read, if-else, case, while, for, date, stty, bc, chmod, mv, head, tail, grep, sed and pipes. It does not use arrays, functions, colours or any database.

Key Features

- 1. File-based storage** — Three text files take the place of a database: **data.txt**, **credentials.txt** and **receipt.txt**.
- 2. Login checked through the file** — The username and password typed by the customer must match a full line in credentials.txt. Only 3 attempts are allowed.
- 3. Duplicate check on registration** — A new customer cannot use a username or mobile number that is already in the files.
- 4. Automatic bill** — Price × Quantity + 18% GST is calculated automatically using **bc**.
- 5. Receipts saved and shown** — Every payment adds one line to receipt.txt with an automatic receipt number. A customer can view all their receipts and the total amount spent.
- 6. Password safety** — The password is hidden while typing (**stty -echo**) and credentials.txt can be read only by its owner (**chmod 600**).

Files Used (Instead of a Database)

Each file stores one record per line. The fields in a line are separated by a colon (:), the same way Linux stores its users in the `/etc/passwd` file.

No.	File	What it stores	Format of one line
1	<code>data.txt</code>	Customer data (details)	<code>username:name:mobile:business</code>
2	<code>credentials.txt</code>	Customer credentials (login)	<code>username:password</code>
3	<code>receipt.txt</code>	Receipts of payments	<code>receipt_no:username:date:service:qty:amount:gst:total:mode</code>

Example contents of the files (after the sample run shown in the Output section):

`data.txt`

```
meera:Meera Kulkarni:9822334455:TrendWave Apparel
rahul:Rahul Patil:9876543210:Patil Sweets
priya:Priya Shah:9000011111:Shah Bakery
```

`credentials.txt`

```
meera:meera123
rahul:rahul123
priya:shah2026
```

`receipt.txt`

```
R1001:meera:13-09-2026:Post Design:3:900:162:1062:UPI
R1002:priya:28-09-2026:Ad Campaign:2:16000:2880:18880:UPI
```

How the Program Checks the Files

Check	Command used in the script
Is the login correct?	<code>grep -c -x -F "\$user:\$pass" credentials.txt</code>
Is the username already taken?	<code>grep -c "^\$user:" credentials.txt</code>
Is the mobile number already registered?	<code>grep -c ":\$mobile:" data.txt</code>
Is the mobile number 10 digits?	<code>echo "\$mobile" grep -c -x "[0-9]\{10\}"</code>
Get the customer's name	<code>grep "^\$user:" data.txt sed 's:/\n/g' head -2 tail -1</code>
Show only my receipts	<code>grep "^R[0-9]*:\$user:" receipt.txt sed 's:/ /g'</code>
Save a new receipt	<code>echo "\$rno:\$user:...:\$mode" >> receipt.txt</code>
Change the password	<code>sed "s/^\$user:./\$user:\$new/" credentials.txt > temp.txt mv temp.txt credentials.txt</code>

-c counts the matching lines, -x matches the whole line only and -F treats the typed text as plain text (not a pattern). If the count is 0 the details are not in the file.

Working Flow

Start → Create files if missing → Main Menu → Register (check files, save details) or Login (check credentials.txt) → Read details from data.txt → Customer Menu → Select Service & Quantity → Calculate Bill (Amount + 18% GST) → Choose Payment Mode → Save Receipt in receipt.txt → View Receipts → Logout → Exit

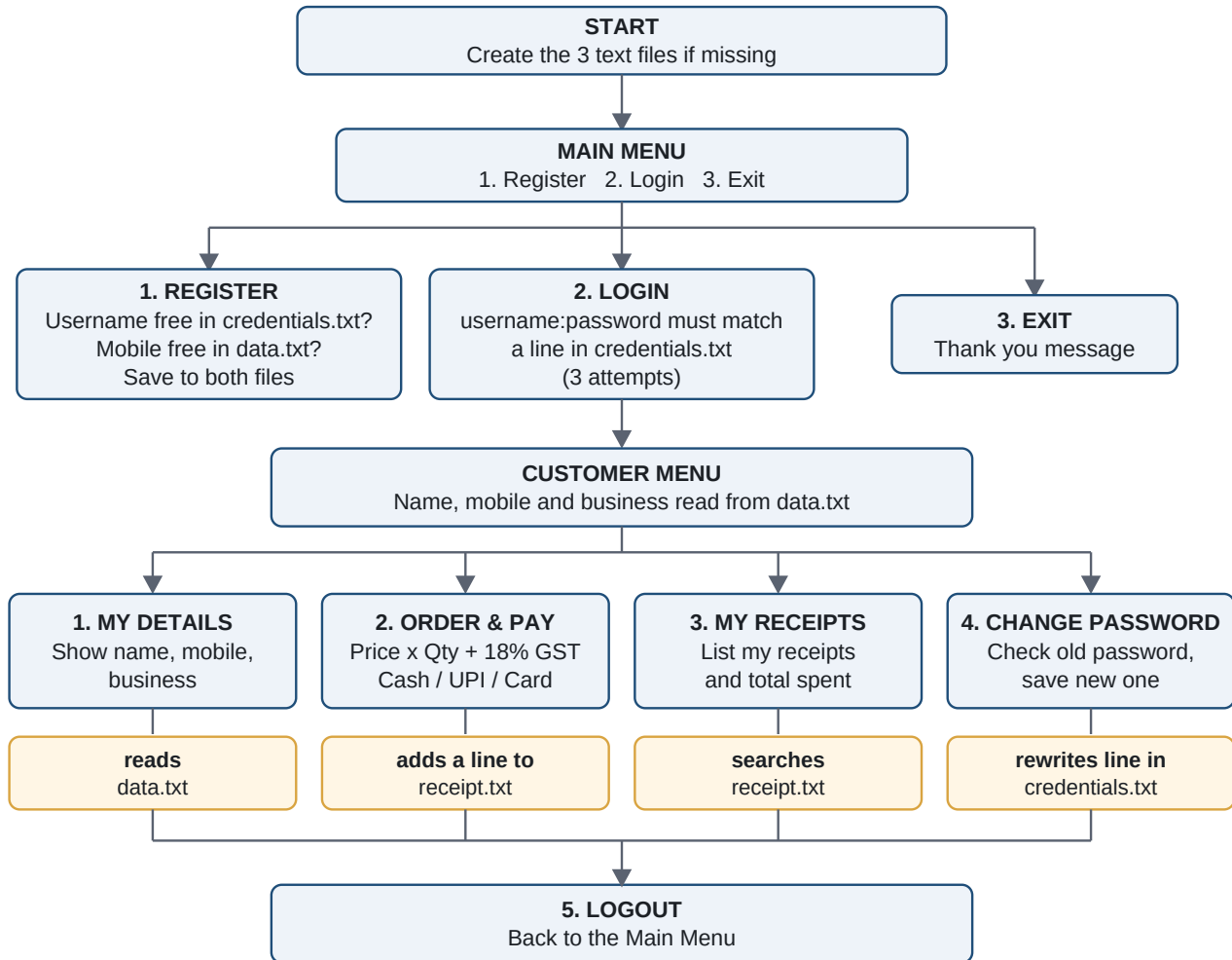


Figure 1: Flow of the program. The orange boxes show which file each option uses.

Services and Prices

No.	Service	Price (Rs.)
1	Social Media Account Setup	1500
2	Custom Post Design (per post)	300
3	Reel / Video Editing	800
4	Instagram Ad Campaign	8000
5	Monthly Management Package	15000

Bill = Price × Quantity; GST = 18% of the bill; Total = Bill + GST.

Source Code (Bash Shell Script) – socialkaro.sh

```
#!/bin/bash
# =====
#  SOCIALKARO - Social Media Marketing Service System (Simple)
#
#  No database is used. Everything is kept in 3 text files:
#    data.txt      -> customer details   username:name:mobile:business
#    credentials.txt -> login details     username:password
#    receipt.txt   -> receipts           receipt_no:username:date:service:qty:amount:gst:total:mode
# =====

DATA=data.txt
CRED=credentials.txt
RECEIPT=receipt.txt

# ----- create the files if they are missing -----
if [ ! -f $DATA ]
then
    echo -n > $DATA
fi
if [ ! -f $CRED ]
then
    echo -n > $CRED
fi
if [ ! -f $RECEIPT ]
then
    echo -n > $RECEIPT
fi
chmod 600 $CRED                # only the owner can read/write passwords

choice=0
while [ "$choice" != "3" ]
do
    echo
    echo "=====
    echo "          SOCIALKARO - MARKETING SERVICES          "
    echo "=====
    echo "Date : $(date '+%d-%m-%Y %I:%M %p') "
    echo "1. New Customer (Register)"
    echo "2. Login"
    echo "3. Exit"
    printf "Enter your choice : "
    read choice

    case $choice in

    1) # ===== REGISTER =====
        echo
        echo "----- NEW CUSTOMER -----"

        user=""
        while [ -z "$user" ]
        do
            printf "Choose username (3-10 small letters/digits) : "
            read user
            if [ $(echo "$user" | grep -c -x "[a-z0-9]\{3,10\}") -eq 0 ]
            then
                echo "Invalid username."
                user=""
            elif [ $(grep -c "^$user:" $CRED) -gt 0 ]
            then
                echo "Username already exists. Try another."
                user=""
            fi
        done

        name=""
        while [ -z "$name" ]
        do
            printf "Full name : "
            read name
            if [ $(echo "$name" | grep -c -x "[A-Za-z ]\{2,30\}") -eq 0 ]
```

```

        then
            echo "Use letters and spaces only."
            name=""
        fi
    done

    mobile=""
    while [ -z "$mobile" ]
    do
        printf "Mobile number (10 digits) : "
        read mobile
        if [ $(echo "$mobile" | grep -c -x "[0-9]\{10\}") -eq 0 ]
        then
            echo "Mobile number must be exactly 10 digits."
            mobile=""
        elif [ $(grep -c ":$mobile:" $DATA) -gt 0 ]
        then
            echo "This mobile number is already registered."
            mobile=""
        fi
    done

    business=""
    while [ -z "$business" ]
    do
        printf "Business name : "
        read business
        if [ $(echo "$business" | grep -c -x "[A-Za-z0-9 ]\{2,30\}") -eq 0 ]
        then
            echo "Use letters, digits and spaces only."
            business=""
        fi
    done

    pass=""
    while [ -z "$pass" ]
    do
        printf "Choose password (min 4 letters/digits) : "
        stty -echo          # hide the password while typing
        read pass
        stty echo
        echo
        if [ $(echo "$pass" | grep -c -x "[A-Za-z0-9]\{4,\}") -eq 0 ]
        then
            echo "Password must be at least 4 letters/digits."
            pass=""
        fi
    done

    echo "$user:$name:$mobile:$business" >> $DATA
    echo "$user:$pass" >> $CRED
    echo "Registration successful! You can login now."
    ;;

2) # ===== LOGIN =====
echo
echo "----- CUSTOMER LOGIN -----"
tries=3
login=no
while [ $tries -gt 0 ]
do
    printf "Username : "
    read user
    printf "Password : "
    stty -echo
    read pass
    stty echo
    echo
    # check the typed details against credentials.txt
    if [ $(grep -c -x -F "$user:$pass" $CRED) -eq 1 ]
    then
        login=yes
        tries=0
    else

```

```

        tries=$(echo "$tries - 1" | bc)
        echo "Wrong username or password. Attempts left : $tries"
    fi
done

if [ $login = no ]
then
    echo "Too many wrong attempts. Access denied."
    continue
fi

# read this customer's details from data.txt
# (turn every : into a new line, then pick the line we need)
line=$(grep "^$user:" $DATA)
name=$(echo "$line" | sed 's:/\n/g' | head -2 | tail -1)
mobile=$(echo "$line" | sed 's:/\n/g' | head -3 | tail -1)
business=$(echo "$line" | sed 's:/\n/g' | tail -1)
echo "Login successful. Welcome, $name!"

opt=0
while [ "$opt" != "5" ]
do
    echo
    echo "----- CUSTOMER MENU -----"
    echo "1. My Details"
    echo "2. Place Order & Pay"
    echo "3. My Receipts"
    echo "4. Change Password"
    echo "5. Logout"
    printf "Enter your choice : "
    read opt

    case $opt in

        1) # ----- MY DETAILS -----
            echo
            echo "Username : $user"
            echo "Name      : $name"
            echo "Mobile   : $mobile"
            echo "Business : $business"
            ;;

        2) # ----- PLACE ORDER -----
            echo
            echo "----- OUR SERVICES -----"
            echo "1. Social Media Account Setup      Rs. 1500"
            echo "2. Custom Post Design (per post)   Rs. 300"
            echo "3. Reel / Video Editing            Rs. 800"
            echo "4. Instagram Ad Campaign           Rs. 8000"
            echo "5. Monthly Management Package      Rs. 15000"
            printf "Select service (1-5) : "
            read sid
            case $sid in
                1) service="Account Setup"; price=1500 ;;
                2) service="Post Design"; price=300 ;;
                3) service="Reel Editing"; price=800 ;;
                4) service="Ad Campaign"; price=8000 ;;
                5) service="Monthly Package"; price=15000 ;;
                *) echo "Invalid service number."
                   continue ;;
            esac

            printf "Quantity : "
            read qty
            if [ $(echo "$qty" | grep -c -x "[1-9][0-9]\{0,2\}") -eq 0 ]
            then
                echo "Quantity must be a number from 1 to 999."
                continue
            fi

            amount=$(echo "$price * $qty" | bc)
            gst=$(echo "$amount * 18 / 100" | bc)
            total=$(echo "$amount + $gst" | bc)
            echo "Amount      : Rs. $amount"
    esac
done

```

```

echo "GST (18%) : Rs. $gst"
echo "Total      : Rs. $total"

echo "Payment mode : 1. Cash   2. UPI   3. Card   4. Cancel"
printf "Select : "
read pm
case $pm in
1) mode=Cash ;;
2) mode=UPI ;;
3) mode=Card ;;
*) echo "Order cancelled."
   continue ;;
esac

# next receipt number = number of lines in receipt.txt + 1001
rno=R$(echo "$(grep -c "" $RECEIPT) + 1001" | bc)
rdate=$(date +%d-%m-%Y)
echo "$rno:$user:$rdate:$service:$qty:$amount:$gst:$total:$mode" >> $RECEIPT

echo
echo "=====
echo "                SOCIALKARO - RECEIPT                "
echo "=====
echo "Receipt No : $rno"
echo "Date       : $rdate"
echo "Customer   : $name ($mobile)"
echo "Business   : $business"
echo "-----"
echo "Service    : $service"
echo "Price      : Rs. $price x $qty = Rs. $amount"
echo "GST (18%)  : Rs. $gst"
echo "TOTAL PAID : Rs. $total"
echo "Paid by    : $mode"
echo "=====
echo "Receipt saved in $RECEIPT"
;;

3) # ----- MY RECEIPTS -----
echo
count=$(grep -c "^R[0-9]*:$user:" $RECEIPT)
if [ $count -eq 0 ]
then
    echo "No receipts found."
    continue
fi
echo "RECEIPT | USER | DATE | SERVICE | QTY | AMOUNT | GST | TOTAL | MODE"
grep "^R[0-9]*:$user:" $RECEIPT | sed 's/:/ | /g'

# add up the TOTAL column (last-but-one field) with a for loop
spent=0
for t in $(grep "^R[0-9]*:$user:" $RECEIPT | sed 's/:[A-Za-z]*$//' | sed 's/.*:/' )
do
    spent=$(echo "$spent + $t" | bc)
done
echo "Total receipts : $count          Total spent : Rs. $spent"
;;

4) # ----- CHANGE PASSWORD -----
printf "Old password : "
stty -echo
read old
stty echo
echo
if [ $(grep -c -x -F "$user:$old" $CRED) -eq 0 ]
then
    echo "Wrong password."
    continue
fi
printf "New password (min 4 letters/digits) : "
stty -echo
read new
stty echo
echo
if [ $(echo "$new" | grep -c -x "[A-Za-z0-9]{4,}") -eq 0 ]

```

```
        then
            echo "Password must be at least 4 letters/digits."
            continue
        fi
        # replace this user's line in credentials.txt
        sed "s/^$user:./$user:$new/" $CRED > temp.txt
        mv temp.txt $CRED
        chmod 600 $CRED
        echo "Password changed successfully."
        ;;

    5) echo "Logged out. Goodbye, $name!" ;;

    *) echo "Invalid choice. Enter 1 to 5." ;;
    esac
done
;;

3) echo "Thank you for using SOCIALKARO!" ;;

*) echo "Invalid choice. Enter 1 to 3." ;;
esac
done
```

Output

Main Menu & New Customer Registration

The mobile number is checked for 10 digits and then searched in data.txt. 9822334455 is refused because it already belongs to another customer. The password is hidden while typing.

```
=====
          SOCIALKARO - MARKETING SERVICES
=====
Date : 28-09-2026  06:20 PM
1. New Customer (Register)
2. Login
3. Exit
Enter your choice : 1

----- NEW CUSTOMER -----
Choose username (3-10 small letters/digits) : priya
Full name : Priya Shah
Mobile number (10 digits) : 98223
Mobile number must be exactly 10 digits.
Mobile number (10 digits) : 9822334455
This mobile number is already registered.
Mobile number (10 digits) : 9000011111
Business name : Shah Bakery
Choose password (min 4 letters/digits) :
Registration successful! You can login now.
```

Customer Login

The first password is wrong, so the program shows the attempts left. The second attempt matches the line in credentials.txt.

```
=====
          SOCIALKARO - MARKETING SERVICES
=====
Date : 28-09-2026  06:20 PM
1. New Customer (Register)
2. Login
3. Exit
Enter your choice : 2

----- CUSTOMER LOGIN -----
Username : priya
Password :
Wrong username or password. Attempts left : 2
Username : priya
Password :
Login successful. Welcome, Priya Shah!
```

CHOICE 1: My Details

The details are read from data.txt.

```
----- CUSTOMER MENU -----
1. My Details
2. Place Order & Pay
3. My Receipts
4. Change Password
5. Logout
Enter your choice : 1

Username : priya
Name      : Priya Shah
Mobile    : 9000011111
Business  : Shah Bakery
```

CHOICE 2: Place Order & Pay

The bill is calculated with bc and the receipt is added to receipt.txt.

```

Enter your choice : 2

----- OUR SERVICES -----
1. Social Media Account Setup      Rs. 1500
2. Custom Post Design (per post)   Rs. 300
3. Reel / Video Editing            Rs. 800
4. Instagram Ad Campaign           Rs. 8000
5. Monthly Management Package      Rs. 15000
Select service (1-5) : 4
Quantity : 2
Amount      : Rs. 16000
GST (18%)   : Rs. 2880
Total       : Rs. 18880
Payment mode : 1. Cash   2. UPI   3. Card   4. Cancel
Select : 2

=====
                SOCIALKARO - RECEIPT
=====
Receipt No : R1002
Date       : 28-09-2026
Customer   : Priya Shah (9000011111)
Business   : Shah Bakery
-----
Service    : Ad Campaign
Price      : Rs. 8000 x 2 = Rs. 16000
GST (18%)  : Rs. 2880
TOTAL PAID : Rs. 18880
Paid by    : UPI
=====
Receipt saved in receipt.txt

```

CHOICE 3: My Receipts

Only this customer's lines are taken from receipt.txt; the totals are added with a for loop.

```

Enter your choice : 3

RECEIPT | USER | DATE | SERVICE | QTY | AMOUNT | GST | TOTAL | MODE
R1002 | priya | 28-09-2026 | Ad Campaign | 2 | 16000 | 2880 | 18880 | UPI
Total receipts : 1      Total spent : Rs. 18880

```

CHOICE 4: Change Password

The old password is checked in credentials.txt, then the line is replaced using sed and mv.

```

Enter your choice : 4
Old password :
New password (min 4 letters/digits) :
Password changed successfully.

```

CHOICE 5: Logout

```

Enter your choice : 5
Logged out. Goodbye, Priya Shah!

```

Wrong Login (3 Attempts)

After 3 wrong passwords the program refuses access.

```
Enter your choice : 2

----- CUSTOMER LOGIN -----
Username : rahul
Password :
Wrong username or password. Attempts left : 2
Username : rahul
Password :
Wrong username or password. Attempts left : 1
Username : rahul
Password :
Wrong username or password. Attempts left : 0
Too many wrong attempts. Access denied.
```

Exit

```
=====
          SOCIALKARO - MARKETING SERVICES
=====
Date : 28-09-2026  06:20 PM
1. New Customer (Register)
2. Login
3. Exit
Enter your choice : 3
Thank you for using SOCIALKARO!
```

Files After the Run

The new customer, the changed password and the new receipt are all saved in the text files. credentials.txt has permission rw----- (600).

```
$ cat data.txt
meera:Meera Kulkarni:9822334455:TrendWave Apparel
rahul:Rahul Patil:9876543210:Patil Sweets
priya:Priya Shah:9000011111:Shah Bakery

$ cat credentials.txt
meera:meera123
rahul:rahul123
priya:shah2026

$ cat receipt.txt
R1001:meera:13-09-2026:Post Design:3:900:162:1062:UPI
R1002:priya:28-09-2026:Ad Campaign:2:16000:2880:18880:UPI

$ ls -l *.txt
-rw----- 1 root root 45 Sep 28 18:20 credentials.txt
-rw-r--r-- 1 root root 132 Sep 28 18:20 data.txt
-rw-r--r-- 1 root root 112 Sep 28 18:20 receipt.txt
```

Linux Commands and Concepts Used

Unit	Topic	How it is used in the project
II – 2.3	File concept, attributes, permission, access methods	Three text files store all records; they are read line by line (sequential access); credentials.txt has permission 600
III – 3.1	echo, printf	Menus, prompts and the receipt
III – 3.1	date	Date on the main menu and on every receipt
III – 3.1	stty	stty -echo hides the password while typing; stty echo shows typing again
III – 3.1	bc	Amount, GST, total, total spent, attempts left and receipt number
III – 3.1	chmod, file permissions	chmod 600 credentials.txt – only the owner can read passwords
III – 3.1	mv	Replaces credentials.txt with the updated copy after a password change
III – 3.1	head, tail	Pick one field (name, mobile, business) out of a line
III – 3.2	Shell variables, read (input statement)	All customer input is read into variables
III – 3.2	if-else, elif	Input checks, duplicate checks and login check
III – 3.2	case (switch statement)	Main menu, customer menu, service price, payment mode
III – 3.2	while, for	Menus and retry loops; for loop adds up the receipt totals
III – 3.2	Pipes ()	grep ... sed ... head ... tail ...
III – 3.3	grep	Searches the three files (login, duplicates, receipts) and checks input format
III – 3.3	sed	Splits a line into fields, formats receipts, changes the password

How to Run

```

$ cd socialkaro
$ chmod +x socialkaro.sh
$ ./socialkaro.sh

Demo logins: meera / meera123      rahul / rahul123

$ script output.txt      # record the session for the report
$ ./socialkaro.sh
$ exit                  # stop recording

```

Objectives

- To provide a simple list of social media marketing services with fixed prices.
- To let customers register and log in with details stored in text files instead of a database.
- To calculate the bill with GST automatically and give a receipt for every payment.
- To practise Linux shell programming: variables, read, if-else, case, while, for, grep, sed and bc.

Advantages

- Very short and simple program that is easy to understand and change.
- No database software is needed – only a Linux terminal.
- Records are plain text files that can be opened with cat, more, grep or tail.
- Automatic calculation avoids mistakes in the bill.
- Login details and duplicate customers are checked through the files.
- The password is hidden while typing and the credentials file is protected with chmod 600.

Disadvantages

- Command-line interface only.
- Passwords are stored as plain text (not encrypted).
- Payments are simulated; there is no real payment gateway.
- Only one service can be ordered at a time.
- Text files are not suitable for a very large number of customers or many users at the same time.

Future Scope

- Store passwords in encrypted (hashed) form.
- Allow several services in one order.
- Add an admin login to add or edit services and see all receipts.
- Move from text files to a database when the number of customers grows.
- Send the receipt to the customer by SMS or email.
- Build a web or mobile version of SOCIALKARO.

Conclusion

SOCIALKARO shows that a small but useful service-ordering system can be built with only basic Linux commands and shell programming. Three text files take the place of a database: data.txt stores customer details, credentials.txt stores login details and receipt.txt stores every payment. Commands such as grep, sed, head, tail and bc are enough to search, update and calculate over these files. The project covers the shell programming topics of the syllabus and can later be extended into a database-backed web or mobile application.

References

- Claude (Anthropic) – Used for project design, Bash coding assistance, debugging and documentation.
- Class Notes – Used for Bash/Linux concepts and project implementation.
- Linux manual pages (man grep, man sed, man bc, man stty).